

Programación de Objetos Distribuidos

Trabajo Práctico Especial 2

Grupo 1

Integrantes:

Patrick Torlaschi - 62273

Santiago Nartallo - 62208

Timoteo Smart - 62844

Tomás Raiti - 62887

Manual de uso

Desde el directorio base del TP:
(Aclaración: *Donde haya un * será sh o ps1 dependiendo de si es Linux/MacOS o Windows.*)

1. Para ejecutar el build:

./scripts/dev/build.*

2. Para ejecutar nodos del servidor:

./scripts/lib/server-node.*

3. Para ejecutar los clientes que realizaran la N-ésima consulta:

./scripts/lib/queryN.*

Parámetros comunes y obligatorios a todas las consultas:

-Daddresses='xx.xx.xx.xx:XXXX;yy.yy.yy.yy:YYYY;..'
[Lista de direcciones IP de los nodos con sus puertos]

-Dcity='XYZ'
[El dataset con el que se trabajara. Solo NYC y CHI son válidos]

-DinPath='XX'
[Path donde están los archivos de entrada de reclamos y tipos de reclamos]

-DoutPath='YY'
[Path a donde estarán los archivos de salida]

Parámetros adicionales a consultas:
Query2

-Dq='x' donde x *es un real en* $(0,\ 1]$

Query3

-Dw='x' donde x *es un entero en* $[1,\ 12]$

Query4

-Dneighbourhood='STR' donde STR *indica el barrio*

Introducción

En este informe analizaremos la arquitectura y las principales decisiones de diseño de nuestro sistema distribuido con *HazelCast* para el procesamiento de consultas a CSVs; Se analizará la forma en la que se leen los archivos, los mappers, reducers, combiners, collators y clientes implementados, teniendo en cuenta cómo estos elementos influyen en la velocidad de procesamiento. Finalmente, compartiremos los resultados de las pruebas realizadas, junto con propuestas para posibles mejoras o expansiones.

Análisis de componentes

- *Lectura de datos*

Al principio la lectura de datos era completa, es decir, se utilizaban todas las columnas del csv y estas se guardaban en objetos Ej: *CHIRow* y *NYCRow*. Esta metodología resultaba ser poco eficiente, por lo que se optimizó el rendimiento almacenando únicamente los datos necesarios para cada query en particular. En lugar de contar con una clase genérica, se implementó al menos una clase específica para cada *query*. En la clase padre Base se encuentra el método parseCsv, el cual fue optimizado, y creado de forma genérica para poder leer y procesar ambos archivos (CHI, NYC). Este permite mapear y guardar los datos a las clases/mapas correspondientes mediante funciones lambda. Además, se hace uso del método *putAll*, el cual es más performante que múltiples *put* consecutivos.

- *Clientes*

Todos los clientes extienden a la clase *Base* con el fin de minimizar la repetición de código y estandarizar los métodos. Dicha clase administra el parseo de los CSVs, la inicialización y las conexiones. Por otro lado, cada cliente es responsable de realizar las operaciones necesarias para poder tanto interpretar como procesar los datos obtenidos, y de guardar la salida.

- *KeyPredicates*

No se utilizaron *KeyPredicates* en la implementación del TP. Se estudió su utilidad y programó uno para la *query* 4, pero se prefirió realizar filtrado en los clientes.

- *Mappers*

Como las *queries* $\{2n + 1 / n \leq 2, n \in \mathbb{N}_0\}$ requerían de un contador para poder producir el resultado pedido, se decidió crear un Mapa Genérico el cual reduciría la cantidad de código:

GenericCountMapper<T>

El mismo es una implementación de *Mapper<Long, T, T, Long>*, es decir, siempre emite un dato de tipo *T* asociado a un entero.

Por otro lado, para las queries $\{2m / m \leq 2, m \in \mathbb{N}\}$, debido a los requisitos fue necesario crear Mappers distintos, respectivamente:

ComplaintQuadrantMapper, *StreetAverageMapper*

El primero emite una clase del tipo *NeighborhoodQuadrant*, el cual almacena coordenadas asociadas al nombre de un barrio en particular; mientras que el segundo emite una dirección (*Address.toString()*) asociado al tipo de evento sucedido (*String*).

Originalmente, se empezó utilizando emisiones condicionales dentro de los mappers, pero se descubrió empíricamente que hacer el filtrado de las tuplas antes de agregarlas al *IMap* de *HazelCast* producía resultados más rápidos, dado que se ahorra la transmisión por red de datos que no serán utilizados.

- *Combiners*

Se utilizó un *Combiner* genérico para las queries $\{2n + 1 / n \leq 2, n \in \mathbb{N}_0\}$. El resto de las queries no utilizaron esta optimización. El *Combiner* fue el siguiente:

GenericCountCombinerFactory<K>

El mismo es una implementación de *CombinerFactory<K , Long, Long>*, es decir acumula enteros para una llave de clase *K*. Consiste en una simple implementación para acumular los valores emitidos por el Mapper.

- *Reducers*

Se utilizó un *Reducer* genérico para las *queries* $\{2n + 1 / n \leq 2, n \in \mathbb{N}_0\}$. Luego para las restantes, se hizo un reducer acorde al caso. La implementación genérica fue la siguiente:

GenericCountReducerFactory<K>

La clase implementa el *ReducerFactory*<K, *Long*, *Long*>. Este suma los enteros emitidos para una clave de clase *K*.

- *Collators*

Los *Collators* se utilizaron principalmente para ordenar la información según fuese necesario y reestructurarla (como en las queries $\{2n + 1 / n \leq 2, n \in \mathbb{N}\}$). Por ejemplo, en la *query* 5, se utilizó para acumular anualmente los resultados de cada mes.

En cuanto al *Collator* de la *query* 3, se optó por agrupar primero los reclamos abiertos por agencia y luego por año, haciendo uso de un *TreeMap*. Esto permite que se tenga naturalmente un orden alfabético entre las agencias (que dado el criterio de Java, no es case-sensitive) y cronológico de años y meses, según lo especificado en el enunciado. Durante el cálculo de la media móvil, se va sumando el total de los reclamos de los meses anteriores incluido el actual, restando cuando la ventana excede el tamaño determinado por la variable w. De esta manera, para los primeros meses del año, cuando aún no están completos los w meses previos, el promedio se calcula considerando solamente los que están disponibles. La implementación de este *Collator* tiene en cuenta tanto el tamaño de la venta como la posibilidad de considerar el número 0 en caso de que no se encuentren reclamos abiertos en determinados meses. Por último, el resultado obtenido se formatea truncando con dos decimales, y se añade a la salida en el orden pedido.

El resto de las *queries* utilizan *Collators* simples, ordenando los resultados según lo especificado en el enunciado.

- *Otros*

Para optimizar el filtrado por barrio de la *query* 4 se creó la clase *HeuristicNeighbourhood*, la cual, conociendo el dominio del input, realiza una

comparación incompleta o heurística de la entrada para no tener que hacer un String.equals() completo sobre cada línea del CSV, ahorrando millones de iteraciones.

Pruebas realizadas

Es importante aclarar que toda prueba realizada en el tp utilizó los csv's completos de NYC y CHI provistos por la cátedra. El de NYC teniendo 32.180.598 registros y el de CHI teniendo 11.100.190 registros. Además, cada nodo se inicializó con 5GB de memoria.

Para la toma de tiempos, se colocó un *logger.info()* con un timestamp al comienzo del parseo de los csv's, al final del parseo y luego antes de empezar a escribir el csv con los resultados. Tomando el tiempo entre el comienzo de parseo y el final del parseo del csv como el *tiempo de lectura* y el tiempo entre el final del parseo y el comienzo de la escritura del csv de resultados como el *tiempo del MapReduce*.

Para el análisis de resultados, primero se realizaron múltiples ejecuciones por *query*, promediando los valores obtenidos. En los resultados los mismos se muestran como un par de curvas, teniendo en el eje x la cantidad de nodos utilizados y en el eje y el tiempo total transcurrido.

Los primeros gráficos comparan la duración entre el parseo del CSV y del *MapReduce*, utilizando distintas escalas para cada línea a fin de comparar el impacto que tiene el número de nodos. No se busca comparar directamente los valores.

Por otro lado, también se realizaron pruebas comparando los efectos del *Combiner* para las *queries* $\{2n + 1 / n \leq 2, n \in \mathbb{N}\}$. En las fig 1.9 a 1.12 se pueden observar los resultados de las mismas y observar las diferencias en los tiempos de MapReduce influenciado por la presencia de un combiner.

Resultados

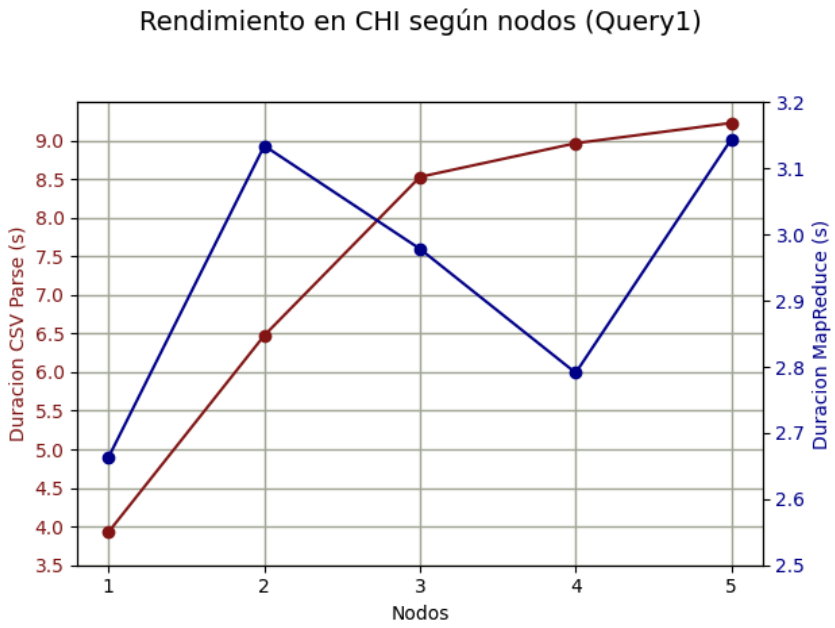


Fig 1.1 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago para la query1 con Combiner

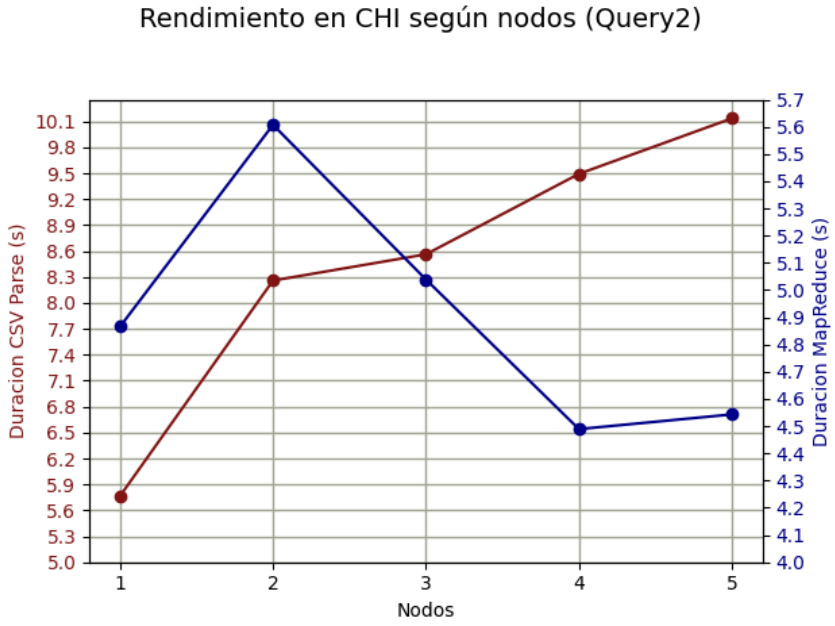


Fig 1.2 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago

para la query2 con Combiner

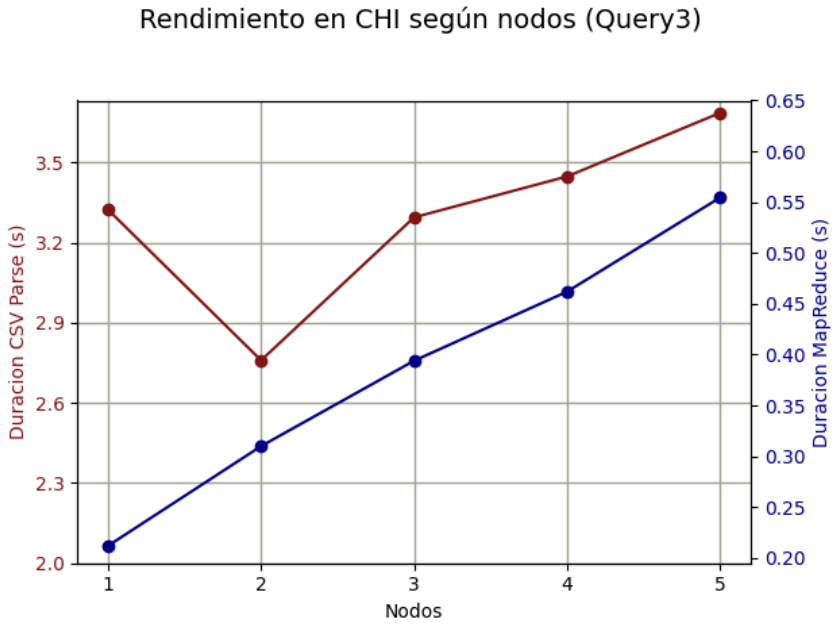


Fig 1.3 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago

para la query3 con Combiner

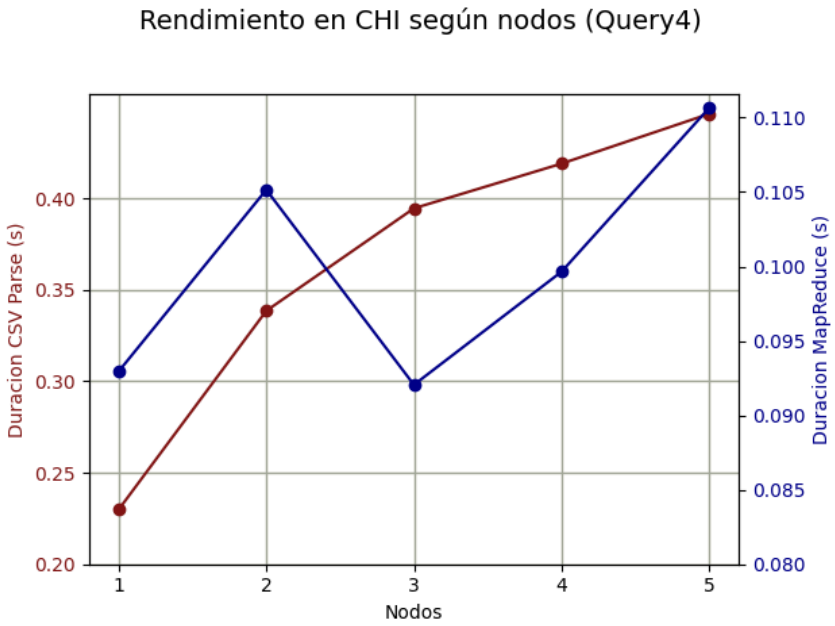


Fig 1.4 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago

para la query4 con Combiner

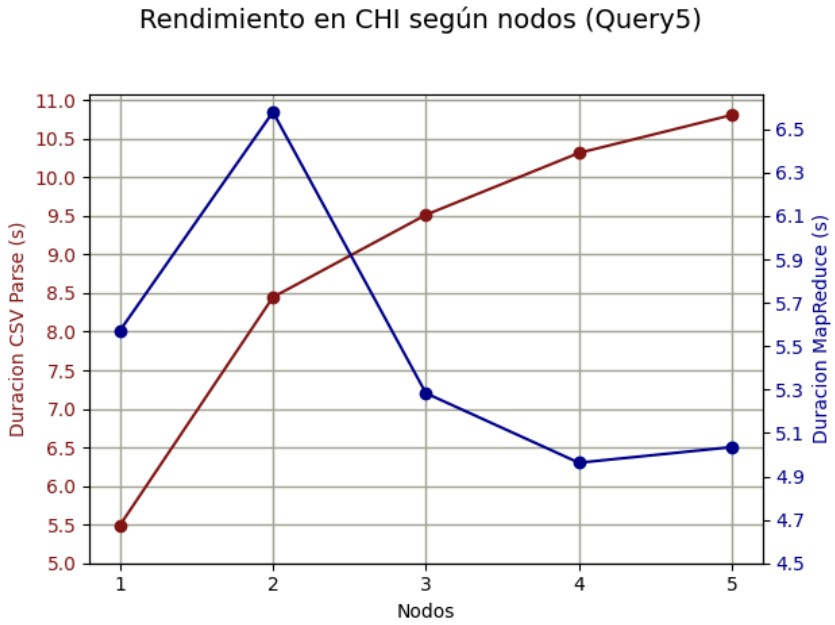


Fig 1.5 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago

para la query5 con Combiner

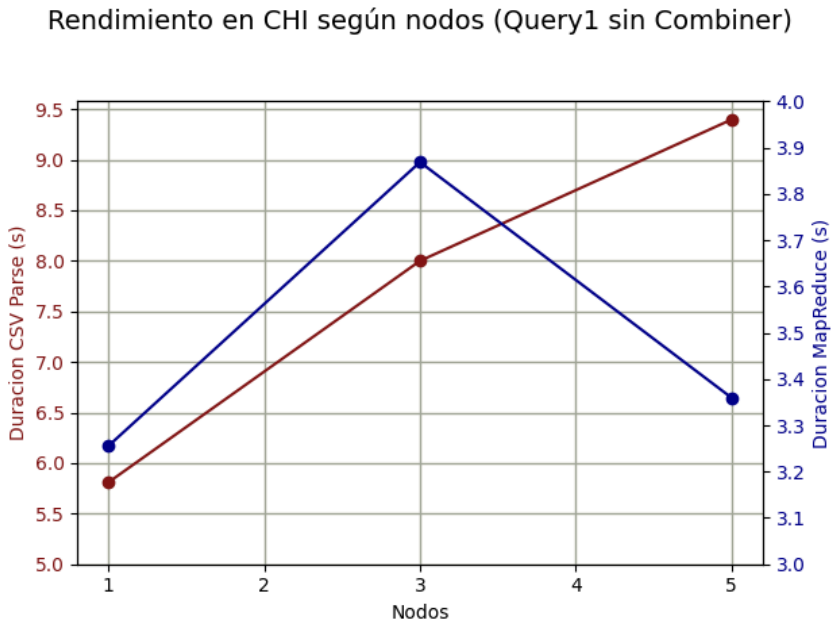


Fig 1.6 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago

para la query1 sin el uso de un Combiner

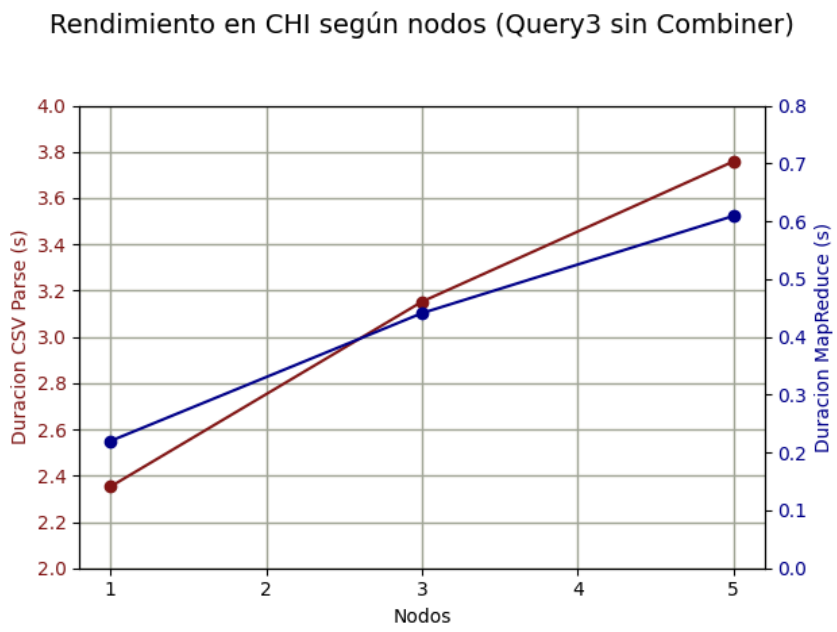


Fig 1.7 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago para la query3 sin el uso de un Combiner

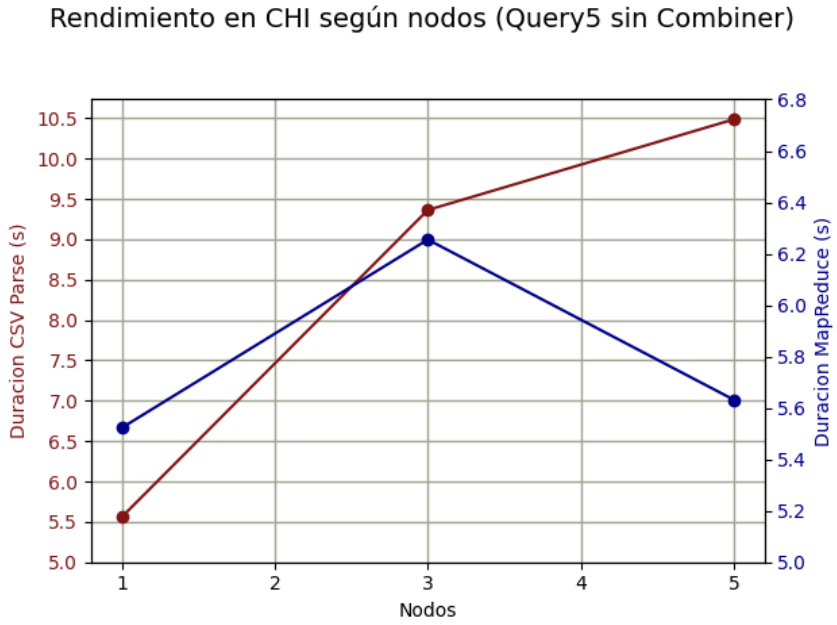


Fig 1.8 Gráfico comparando el rendimiento entre el parse y el MapReduce del csv de Chicago para la query5 sin el uso de un Combiner

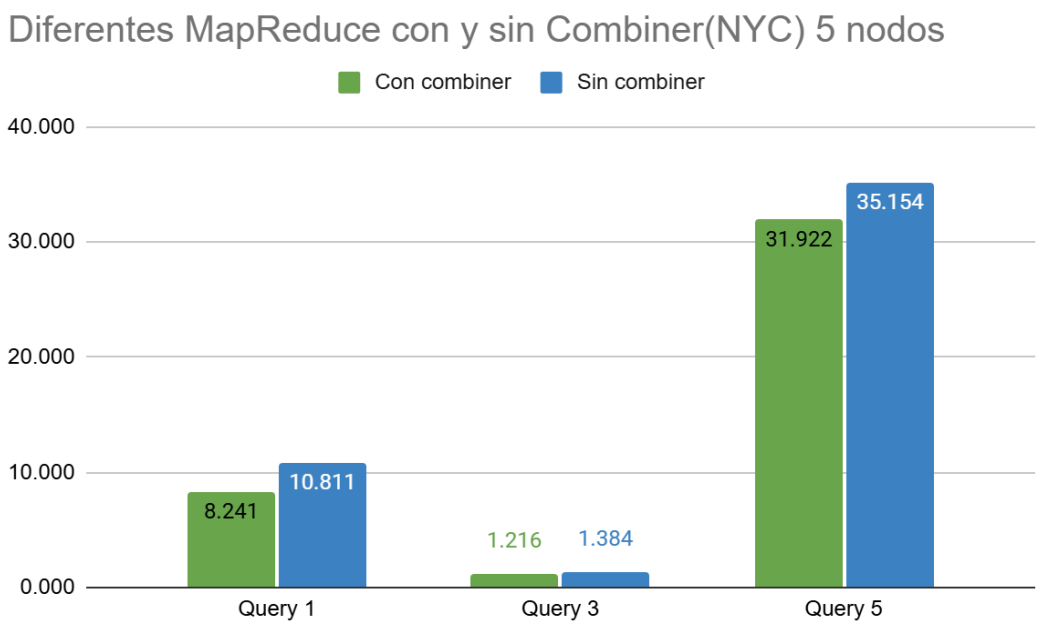


Fig 1.9 Gráfico comparando tiempo de mapReduce en las queries 1, 3 y 5 con 5 nodos en NYC

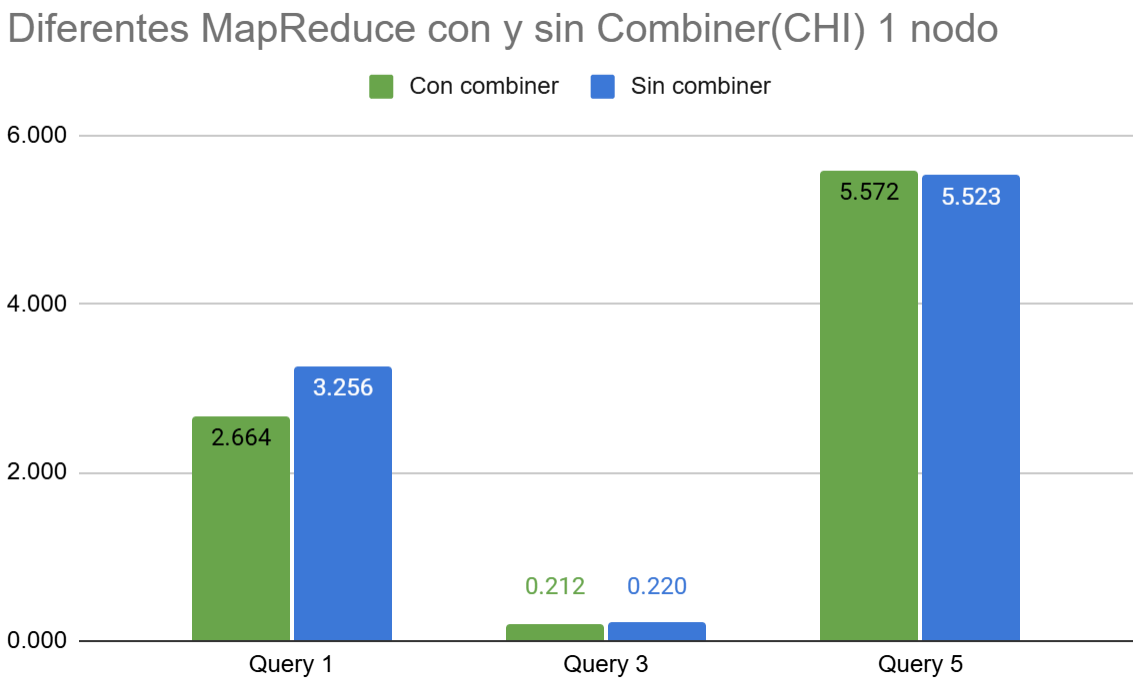


Fig 1.10 Gráfico comparando tiempo de mapReduce en las queries 1, 3 y 5 con 1 nodo en CHI

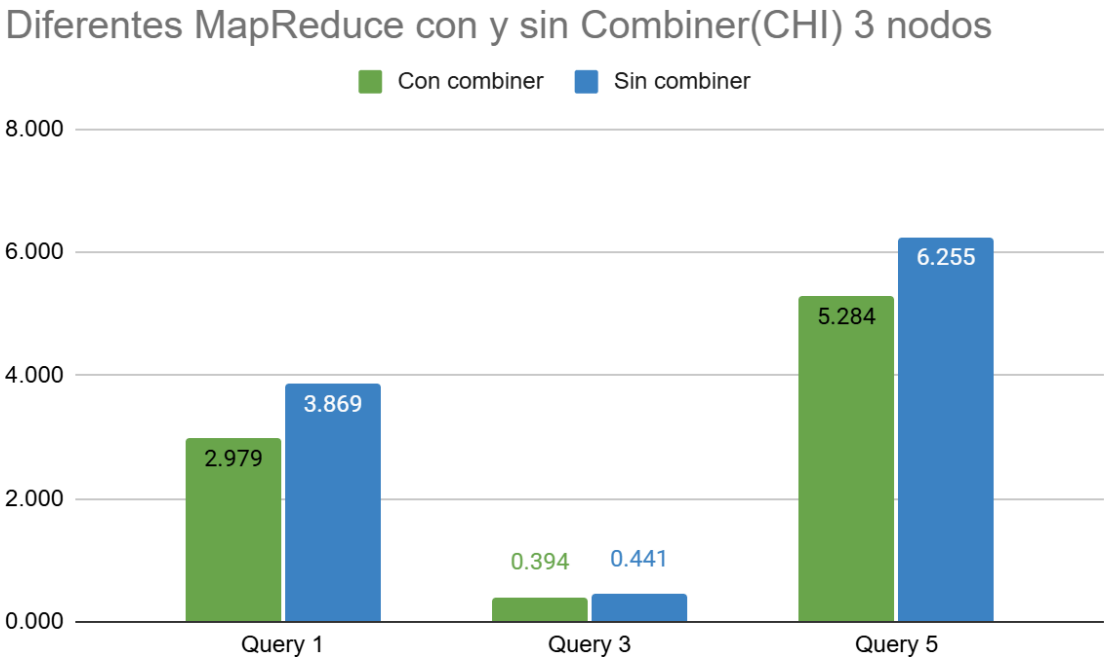


Fig 1.11 Gráfico comparando tiempo de mapReduce en las queries 1, 3 y 5 con 3 nodos en CHI

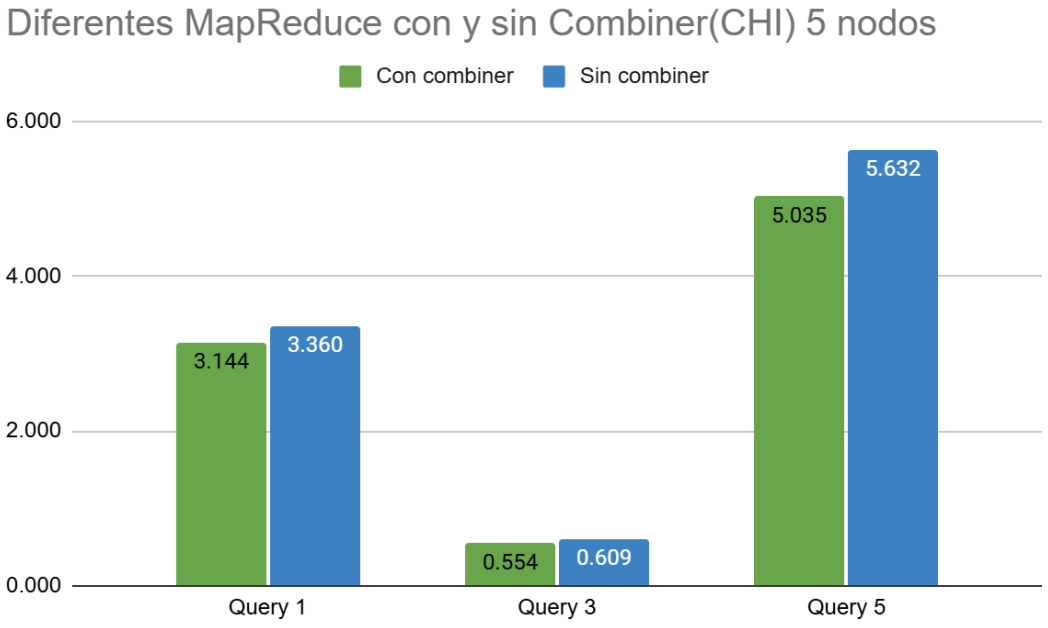


Fig 1.12 Gráfico comparando tiempo de mapReduce en las queries 1, 3 y 5 con 5 nodos en CHI

Análisis de resultados

Como se puede notar, la gran mayoría de las *queries* presentan una tendencia similar en la relación entre el número de nodos y el tiempo de cada paso de la ejecución.

A mayor número de servidores, el tiempo de parseo del archivo aumenta, lo que se aproxima a las estimaciones iniciales hechas por el grupo. Esto se debe al overhead que implica distribuir los resultados del hashmap entre múltiples servidores.

Por el contrario, se esperaba que a mayor números de nodos, el map reduce se beneficiaría inversamente, dado que contaría con más servidores para distribuir la tarea. Sin embargo, las mismas generaron resultados inusuales, con picos inesperados en 2 y 5 nodos. Se cree que esto fue causado por correr los servidores dentro de una misma computadora y no distribuirlo en la red, lo cual desafortunadamente no se pudo probar por problemas de coordinación de tiempos dentro del equipo.

Se puede observar en promedio una reducción entre el 10% y 23% del tiempo en realizar el MapReduce utilizando un combiner en las queries $\{2n + 1 \,/\, n \leq 2,\ n \in \mathbb{N}\}$. Lo cual era esperado, pues evita emitir valores de más al acumular los valores para una llave antes de emitirlos.

Posibles mejoras o expansiones

En un futuro se podrían realizar más pruebas comparando los efectos de distribuir la red en distintas computadoras, incluso dentro de una misma red y redes distintas. En un principio, las mismas fueron realizadas, y todo funcionó como se esperaba, pero no fueron cronometradas en su momento por lo que no se pudieron utilizar como métricas. El impacto del combiner en la performance para estos casos debería verse aún más reflejado, ya que reduciría la saturación en la red.

También se estudió mejorar la performance de la *query* 5, la cual recibió la menor cantidad de optimizaciones durante el desarrollo. Se podría intentar

plantear un MapReduce que permita hacer la acumulación de los resultados de forma distribuída, lo que bajaría la carga sobre el collator del cliente.

Por último, se notó que la query 4 con parámetro QUEENS termina el parseo del csv en tiempo razonable, pero por razones desconocidas, el *MapReduce* nunca finaliza antes del *shutdown* de 20 minutos. Se testeó con un archivo de prueba más corto, y el código se ejecuta a la perfección, y los otros barrios de NYC funcionan con el archivo completo en tiempos menores a 1 minuto. Desafortunadamente, no se pudo descubrir la naturaleza de este *bug*.

Apéndice

Query 1 sin combiner				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	5.811	3.256	9.067
CHI	3	8.000	3.869	11.869
CHI	5	9.397	3.360	12.757
NYC	5	23.391	10.811	34.203

Tabla 1 - Tiempos de ejecución query 1 con 1, 3, 5 nodos sin combiner

Query 3 sin combiner (w=3)				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	2.353	0.220	2.573
CHI	3	3.152	0.441	3.593
CHI	5	3.758	0.609	4.367
NYC	5	13.215	1.384	14.600

Tabla 2 - Tiempos de ejecución query 3 con 1, 3, 5 nodos sin combiner

Query 5 sin combiner				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	5.561	5.523	11.084
CHI	3	9.362	6.255	15.617
CHI	5	10.485	5.632	16.117
NYC	5	27.862	35.154	63.016

Tabla 3 - Tiempos de ejecución query 5 con 1, 3, 5 nodos sin combiner

Query 1				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	3.931	2.6635	6.595
CHI	2	6.475	3.1345	9.6095
CHI	3	8.528	2.979	11.506
CHI	4	8.964	2.791	11.755
CHI	5	9.227	3.144	12.371
NYC	5	24.657	8.128	32.786

Tabla 4 - Tiempos de ejecución promedio query 1 con 1 a 5 nodos con combiner

Query 2 (q=0.3)				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	5.766	8.185	13.951
CHI	2	8.259	5.607	13.866
CHI	3	8.563	5.039	13.601
CHI	4	9.493	4.489	13.982
CHI	5	10.135	4.543	14.678
NYC	5	36.227	22.815	59.042

Tabla 5 - Tiempos de ejecución promedio query 2 con 1 a 5 nodos con combiner

Query 3 (w=3)				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	3.322	0.212	3.534
CHI	2	2.759	0.310	3.069
CHI	3	3.294	0.394	3.688
CHI	4	3.447	0.462	3.909
CHI	5	3.685	0.554	4.240
NYC	5	13.215	1.216	14.432

Tabla 6 - Tiempos de ejecución promedio query 3 con 1 a 5 nodos con combiner

Query 4 (neighborhood=OHARE)				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	2.660	1.076	3.736
CHI	2	3.917	1.217	5.134
CHI	3	4.565	1.065	5.631
CHI	4	4.848	1.153	6.002
CHI	5	5.163	1.281	6.444
NYC	5	11.547	10.186	21.734

Tabla 7 - Tiempos de ejecución promedio query 4 con 1 a 5 nodos con combiner

Query 5				
Location	Nodes	CSV Parse (s)	MapReduce (s)	Total (s)
CHI	1	5.494	5.572	11.067
CHI	2	8.447	6.579	15.027
CHI	3	9.514	5.284	14.797
CHI	4	10.315	4.963	15.278
CHI	5	10.807	5.035	15.842
NYC	5	28.373	31.922	60.295

Tabla 8 - Tiempos de ejecución promedio query 5 con 1 a 5 nodos con combiner

Primeras estimaciones de performance según el número de nodos.

